UNIT I:

- **Introduction:** Evolution, Software development projects, Exploratory style of software developments, Emergence of software engineering, Notable changes in software development practices, Computer system engineering.
- **Software Life Cycle Models:** Basic concepts, Waterfall model and its extensions, Rapid application development, Agile development model, Spiral model.

UNIT I:

- **Introduction:** Evolution, Software development projects, Exploratory style of software developments, Emergence of software engineering, Notable changes in software development practices, Computer system engineering.
- **Software Life Cycle Models:** Basic concepts, Waterfall model and its extensions, Rapid application development, Agile development model, Spiral model.

Evolution

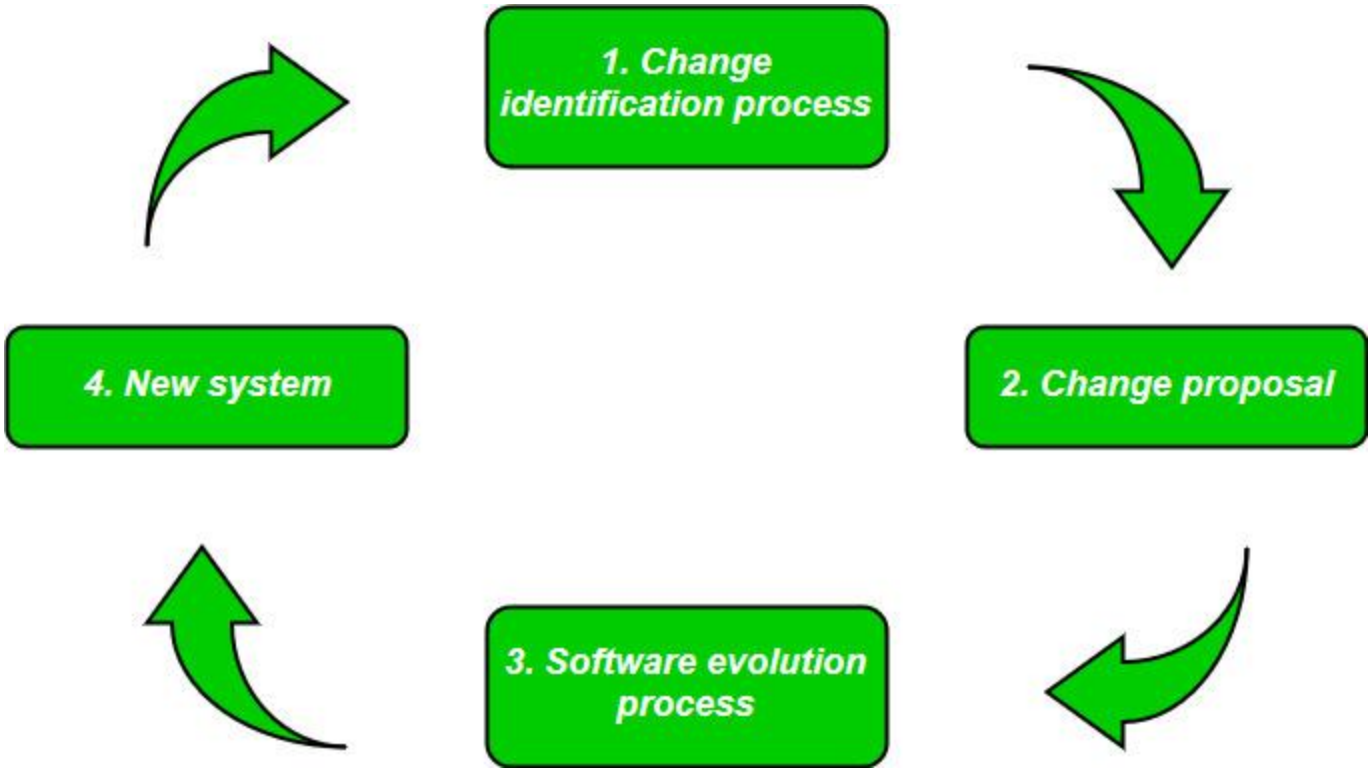
- Software Evolution is a term that refers to the process of developing software initially, and then timely updating it for various reasons, i.e., to add new features or to remove obsolete functionalities, etc.
- **Necessity of Software Evolution**
- **Laws used for Software Evolution**

Evolution

- Software Evolution is a term that refers to the process of developing software initially, and then timely updating it for various reasons, i.e., to add new features or to remove obsolete functionalities, etc.
- **Necessity of Software Evolution**
- **Laws used for Software Evolution**

Evolution

- **Necessity of Software Evolution:**



Evolution

- **Change in requirement with time:** With time, the organization's needs and modus Operandi of working could substantially be changed so in this frequently changing time the tools(software) that they are using need to change to maximize the performance.
- **Errors and bugs:** As the age of the deployed software within an organization increases their preciseness or impeccability decrease and the efficiency to bear the increasing complexity workload also continually degrades.

Evolution

- **Security risks:** Using outdated software within an organization may lead you to at the verge of various software-based cyber attacks and could expose your confidential data illegally associated with the software that is in use.
- **For having new functionality and features:** In order to increase the performance and fast data processing and other functionalities, an organization need to continuously improve the software throughout its life cycle.

Evolution

- Software Evolution is a term that refers to the process of developing software initially, and then timely updating it for various reasons, i.e., to add new features or to remove obsolete functionalities, etc.
- **Necessity of Software Evolution**
- **Laws used for Software Evolution**

Evolution

- **Laws used for Software Evolution**
- **1. Law of Continuing Change**
  - This law states that any software system that represents some real-world reality undergoes continuous change.
- **2. Law of Increasing Complexity**
  - As an evolving program changes, its structure becomes more complex unless effective efforts are made to avoid this phenomenon.

UNIT I:

- **Introduction:** Evolution, Software development projects, Exploratory style of software developments, Emergence of software engineering, Notable changes in software development practices, Computer system engineering.
- **Software Life Cycle Models:** Basic concepts, Waterfall model and its extensions, Rapid application development, Agile development model, Spiral model.

Software development projects

- Web Development Projects: E-commerce Websites, Content Management Systems (CMS), Web Applications.
- Mobile App Development Projects: iOS Apps, Android Apps, Cross-Platform Apps.
- Enterprise Software Development Projects : Customer Relationship Management (CRM) Systems, Enterprise Resource Planning (ERP) Systems, Business Intelligence (BI) Tools.

Software development projects

- Game Development Projects: Console Games, PC Games, Mobile Games.
- Embedded Systems Development Projects: IoT Devices, Automotive Software, Medical Devices.
- Open-Source Projects: Libraries and Frameworks, Tools and Utilities.
- Artificial Intelligence (AI) and Machine Learning (ML) Projects: Natural Language Processing (NLP) Applications, Predictive Analytics, Computer Vision.

UNIT I:

- **Introduction:** Evolution, Software development projects, Exploratory style of software developments, Emergence of software engineering, Notable changes in software development practices, Computer system engineering.
- **Software Life Cycle Models:** Basic concepts, Waterfall model and its extensions, Rapid application development, Agile development model, Spiral model.

Exploratory style of software developments

- Exploratory program development style refers to an informal development style in which the programmer uses his own intuition to develop a program rather than making use of the systematic body of knowledge which is categorized under the software engineering discipline.
- This style does not offer any rules to start developing any software.
- This style of software development is only used for the development of small programs. This style is not really used in industries nowadays.

Exploratory style of software developments

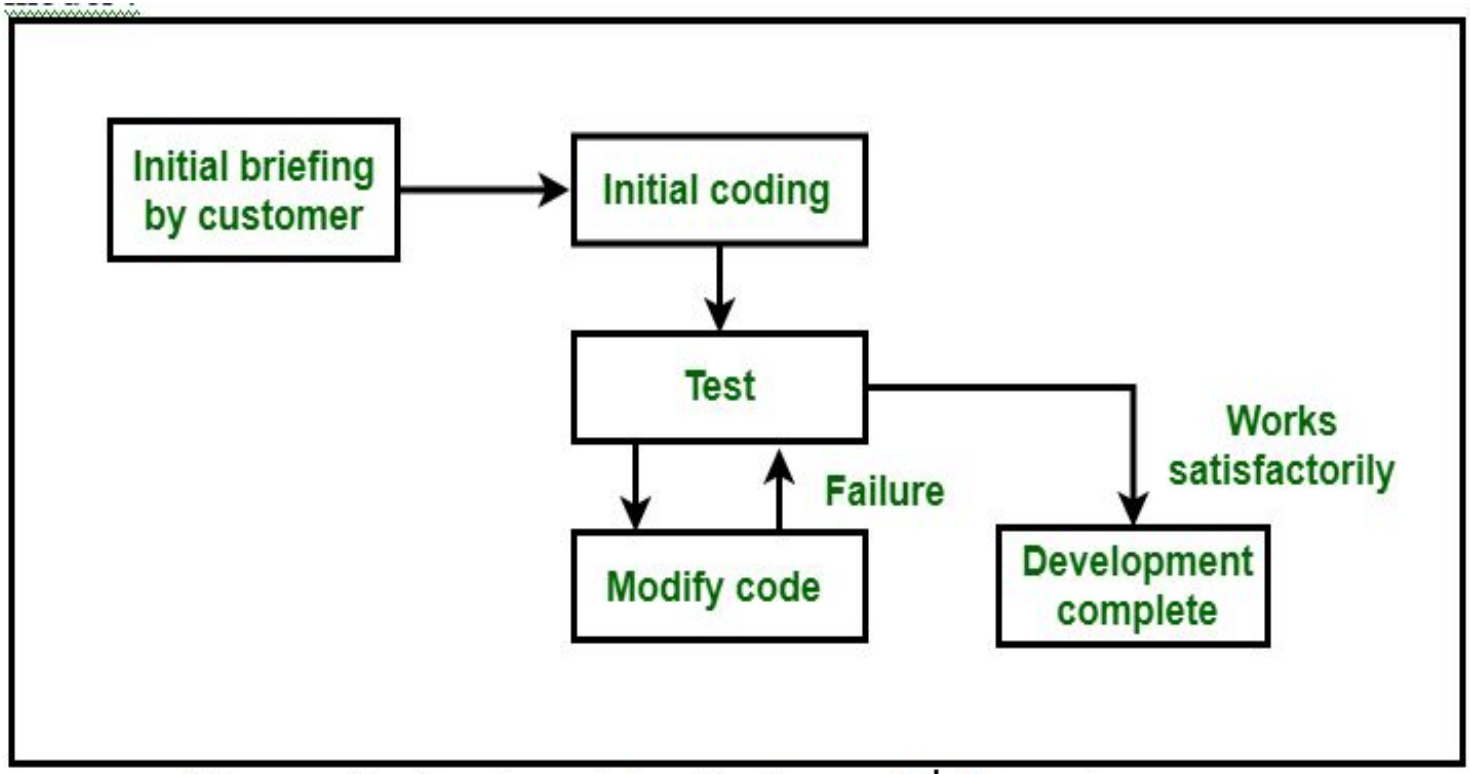


Figure – Exploratory style of software development

Exploratory style of software developments

What's wrong with this model?

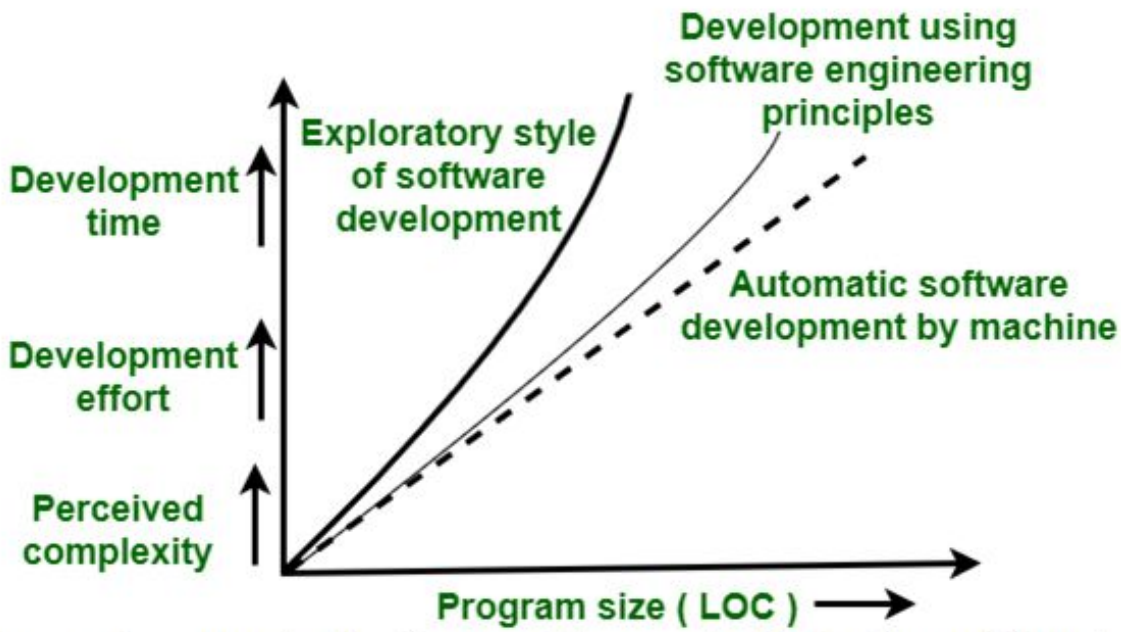


Figure – Increase in development time and effort with problem size.

Exploratory style of software developments

- In the above figure, the **thick line** plots represent the case in which the exploratory style is used to develop a program. As program size increases, required effort and time increase almost exponentially.
- The **thin solid line** is used to represent a case when development is carried out using software engineering principles. In this case, it becomes possible to solve a problem with effort and time that is almost linear in program size.

Exploratory style of software developments

- **Dotted line** is used to represent a case when development is carried out by an automated machine. In this case, an increase in effort and time with size would be even closer to a linear increase with size.

UNIT I:

- **Introduction:** Evolution, Software development projects, Exploratory style of software developments, Emergence of software engineering, Notable changes in software development practices, Computer system engineering.
- **Software Life Cycle Models:** Basic concepts, Waterfall model and its extensions, Rapid application development, Agile development model, Spiral model.

Emergence of software engineering

- Software engineering discipline is the result of advancement in the field of technology.
- Various innovations and technologies that led to the emergence of software engineering discipline are:
- **Early Days (1940s-1960s)**
- **Birth of Software Engineering (Late 1960s)**
- **Growth and Evolution (1970s-1990s)**
- **Modern Era (2000s-Present)**

Emergence of software engineering

- **Early Days (1940s-1960s):**
- **Initial Programming:** The first computers were programmed using assembly language and machine code. This period saw the rise of pioneers like Alan Turing and John von Neumann.
- **Software Crisis:** As software projects grew in complexity, many faced delays, budget overruns, and failures. This era, especially the late 1960s, highlighted the need for more disciplined approaches to software development.

Emergence of software engineering

- **Birth of Software Engineering (Late 1960s)**
- **NATO Conference (1968):** The term "software engineering" was popularized during the NATO Software Engineering Conference, which addressed the growing software crisis. It emphasized applying engineering principles to software development.
- **First Practices:** Early methodologies like the Waterfall Model were introduced to provide a structured approach to software development, breaking it into distinct phases: requirements, design, implementation, testing, deployment, and maintenance.

Emergence of software engineering

- **Growth and Evolution (1970s-1990s)**
- **Structured Programming:** The 1970s saw the rise of structured programming languages like C and Pascal, which promoted clear, modular code.
- **Object-Oriented Programming (OOP):** The 1980s brought about OOP languages like C++ and Smalltalk, which encouraged the use of objects to represent data and behavior, leading to more reusable and maintainable code.
- **Software Development Methodologies:** New methodologies emerged, including Agile, which focused on flexibility, collaboration, and iterative development, and Rapid Application Development (RAD), which emphasized quick prototyping and user feedback.

Emergence of software engineering

- **Modern Era (2000s-Present)**
- **Agile and DevOps:** Agile methodologies became mainstream, promoting iterative development, continuous integration, and delivery. DevOps practices emerged to bridge the gap between development and operations, ensuring faster and more reliable software deployment.
- **Advanced Tools and Technologies:** The rise of sophisticated development tools, version control systems (e.g., Git), and integrated development environments (IDEs) revolutionized how software is developed.

Emergence of software engineering

- **Software as a Service (SaaS):** The advent of cloud computing and SaaS models transformed software distribution and deployment, allowing users to access applications over the internet.
- **Artificial Intelligence and Machine Learning:** AI and ML technologies are now being integrated into software development, enabling smarter and more adaptive applications

UNIT I:

- **Introduction:** Evolution, Software development projects, Exploratory style of software developments, Emergence of software engineering, Notable changes in software development practices, Computer system engineering.
- **Software Life Cycle Models:** Basic concepts, Waterfall model and its extensions, Rapid application development, Agile development model, Spiral model.

Notable changes in software development practices

- **1. Shift from Waterfall to Agile**
  - **Waterfall Model:** Traditional linear and sequential approach with distinct phases (requirements, design, implementation, testing, deployment, maintenance).
  - **Agile Methodology:** Emphasizes flexibility, collaboration, and iterative development with regular feedback loops and continuous improvement.
- **2. Rise of DevOps**
  - **DevOps:** Integrates development and operations teams to streamline the software development lifecycle, improve collaboration, and accelerate delivery. Focuses on continuous integration, continuous delivery, and automation.

Notable changes in software development practices

- **3. Microservices Architecture**
  - **Monolithic Systems:** Single, large application with tightly coupled components.
  - **Microservices:** Breaks down applications into smaller, independent services that can be developed, deployed, and scaled separately, improving flexibility and maintainability.
- **4. Cloud Computing**
  - **On-Premises Infrastructure:** Traditional approach with physical servers and data centers.
  - **Cloud Computing:** Utilizes remote servers hosted on the internet to store, manage, and process data, offering scalability, cost-efficiency, and accessibility.

Notable changes in software development practices

- **5. Low-Code and No-Code Development**
  - **Traditional Coding:** Requires extensive programming knowledge and manual coding.
  - **Low-Code/No-Code Platforms:** Enable developers and non-developers to create applications through visual interfaces and pre-built templates, reducing development time and complexity.
- **6. Artificial Intelligence and Machine Learning**
  - **Traditional Software:** Relying on predefined rules and logic.
  - **AI/ML Integration:** Incorporates machine learning algorithms and AI models to enable predictive analytics, natural language processing, and automated decision-making.

Notable changes in software development practices

- **7. Open Source Software**
  - **Proprietary Software:** Closed-source with restricted access to source code.
  - **Open Source:** Software with publicly accessible source code, encouraging collaboration, innovation, and transparency.
- **8. Security Practices**
  - **Basic Security:** Basic measures like firewalls and antivirus software.
  - **Advanced Security:** Incorporating practices like encryption, regular security audits, and DevSecOps to ensure robust protection against cyber threats.

UNIT I:

- **Introduction:** Evolution, Software development projects, Exploratory style of software developments, Emergence of software engineering, Notable changes in software development practices, Computer system engineering.
- **Software Life Cycle Models:** Basic concepts, Waterfall model and its extensions, Rapid application development, Agile development model, Spiral model.

Computer system engineering

- Computer system engineering involves the design, development, and maintenance of computer systems, including both hardware and software components. It ensures that these systems meet performance, reliability, and usability requirements.
- **Key Components**
- **Process**
- **Challenges**

Computer system engineering

- **Key Components:**
- **Hardware Design:** Selecting and configuring hardware components to optimize performance, power efficiency, and cost.
- **Software Development:** Creating software that interacts seamlessly with hardware, including operating systems, drivers, and application software.
- **System Integration:** Ensuring that all components of the computer system work together harmoniously.

Computer system engineering

- **Networking:** Designing and implementing communication protocols and network architectures to enable data exchange between systems.
- **Security:** Integrating security measures to protect against cyber threats and ensure data integrity and confidentiality.
- **Performance Optimization:** Tuning the system to achieve optimal performance and efficiency.

Computer system engineering

- **Process:**
- **Requirements Analysis:** Understanding the needs and constraints of the system, including functional, performance, and security requirements.
- **System Design:** Creating a blueprint of the system, detailing the architecture and interactions between components.
- **Implementation:** Developing and integrating hardware and software components.
- **Testing and Validation:** Ensuring that the system meets all requirements through rigorous testing and validation.
- **Deployment and Maintenance:** Installing the system and providing ongoing support and updates.

Computer system engineering

- **Challenges:**
- **Complexity:** Managing the intricate interactions between hardware and software components.
- **Scalability:** Designing systems that can grow and adapt to increasing demands.
- **Security:** Protecting systems from evolving cyber threats.
- **Interoperability:** Ensuring that different systems and components can work together seamlessly.

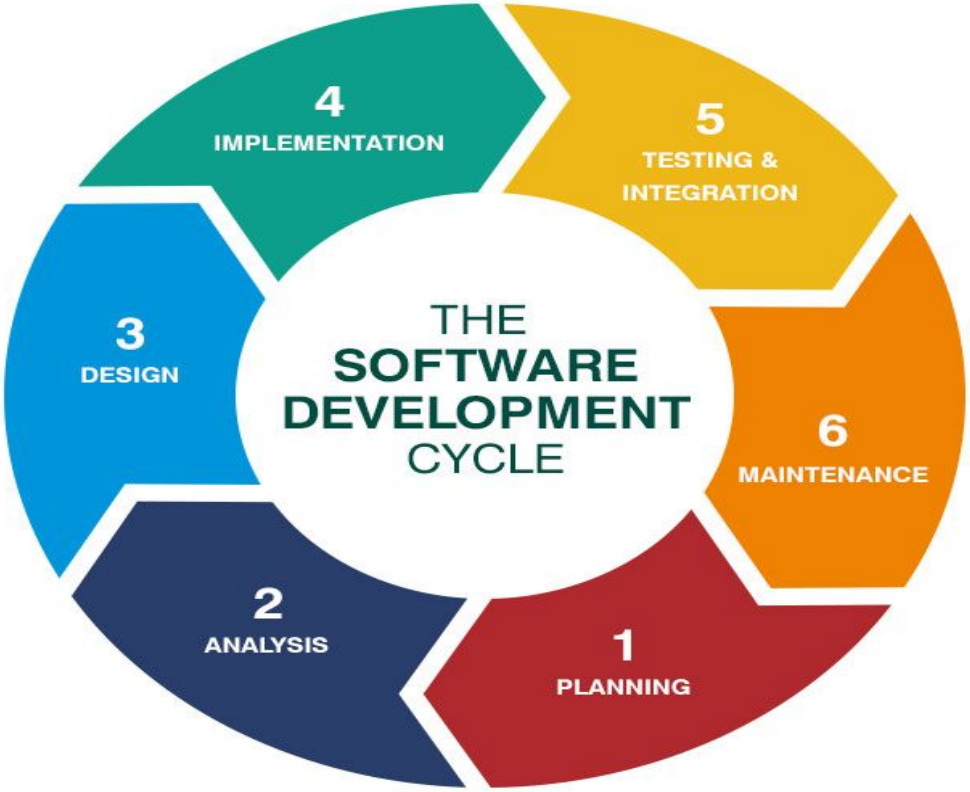
UNIT I:

- **Introduction:** Evolution, Software development projects, Exploratory style of software developments, Emergence of software engineering, Notable changes in software development practices, Computer system engineering.
- **Software Life Cycle Models:** Basic concepts, Waterfall model and its extensions, Rapid application development, Agile development model, Spiral model.

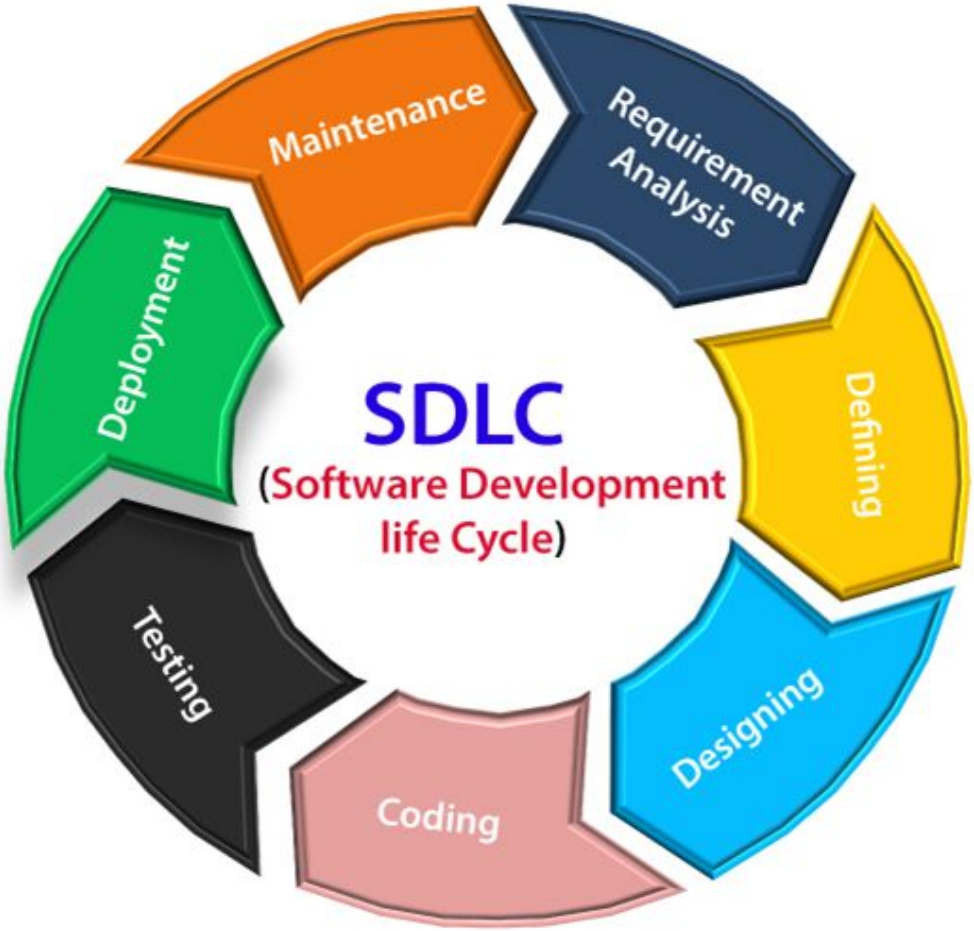
Basic concepts: SDLC

- Software Development Life Cycle (SDLC) is a process used by the software industry to design, develop and test high quality software's.
- The SDLC aims to produce a high-quality software that meets or exceeds customer expectations, reaches completion within times and cost estimates.
- It is also called as Software Development Process.
- ISO/IEC 12207 is an international standard for software life-cycle processes. It aims to be the standard that defines all the tasks required for developing and maintaining software.

Basic concepts: SDLC



Basic concepts: SDLC



Basic concepts: SDLC

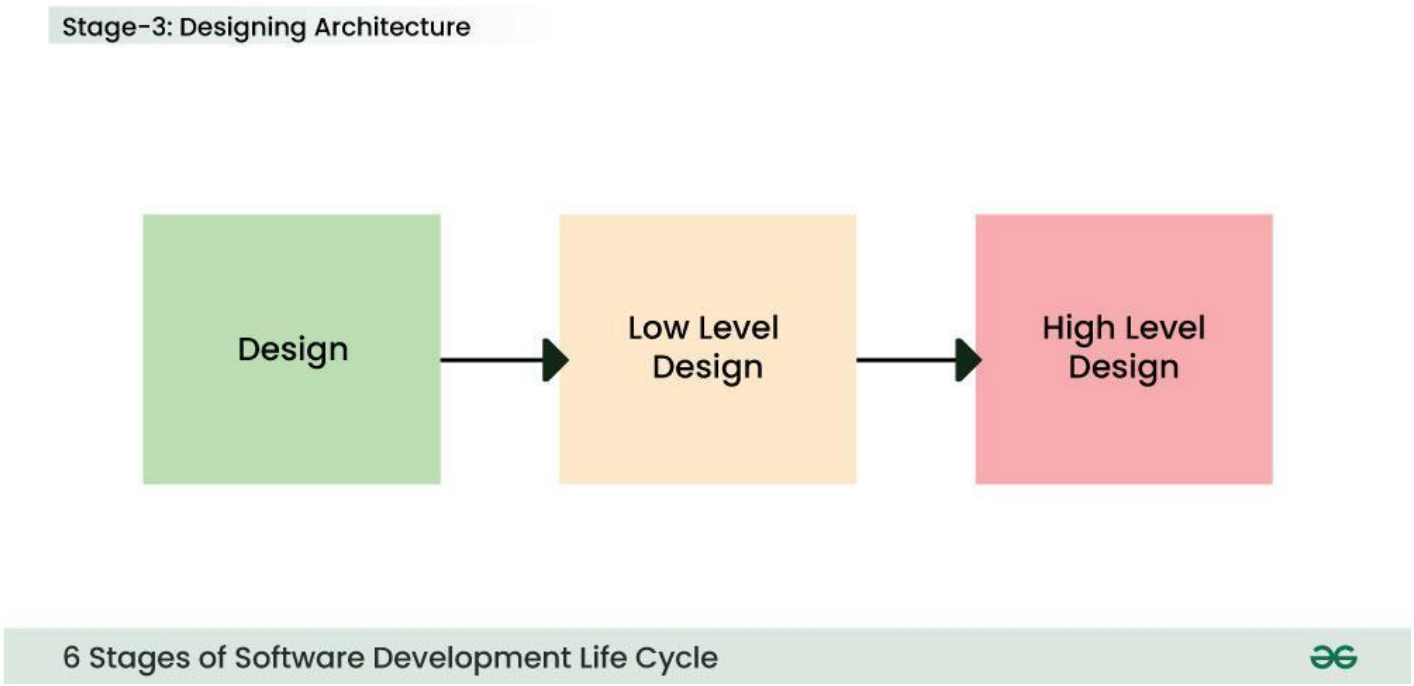
- **Planning and Requirement Analysis**
- Requirement analysis is the most important and fundamental stage in SDLC.
- It is performed by the senior members of the team with inputs from the customer, the sales department, market surveys and domain experts in the industry.
- This information is then used to plan the basic project approach and to conduct product feasibility study in the economical, operational and technical areas.
- Planning for the quality assurance requirements and identification of the risks associated with the project is also done in the planning stage.

Basic concepts: SDLC

- **Defining Requirements**
- Once the requirement analysis is done the next step is to clearly define and document the product requirements and get them approved from the customer or the market analysts.
- This is done through an SRS (Software Requirement Specification) document which consists of all the product requirements to be designed and developed during the project life cycle.

Basic concepts: SDLC

- **Designing:**



Basic concepts: SDLC

- **Designing:**
- In this stage, the requirements gathered in the SRS document is used as information to obtain the software architecture.
- Moreover, this phase also consists of storyboarding the software that is needed for functionality.
- Through this, the developers then create either rough working models, or illustrates how the software will work, how it will look, how usage flows will move from screen to screen, and more.

Basic concepts: SDLC

- **Implementation / Coding:**
- In this stage of SDLC the actual development starts and the product is built.
- The programming code is generated as per design during this stage.
- If the design is performed in a detailed and organized manner, code generation can be accomplished without much hassle.
- Developers must follow the coding guidelines defined by their organization and programming tools.

Basic concepts: SDLC

- **Testing the Product**
- This stage is usually a subset of all the stages as in the modern SDLC models, the testing activities are mostly involved in all the stages of SDLC.
- However, this stage refers to the testing of the product where product defects are reported, tracked, fixed and retested, until the product reaches the quality standards defined in the SRS.

Basic concepts: SDLC

- **Deployment in the Market and Maintenance**
- Once the product is tested and ready to be deployed it is released formally in the appropriate market.
- The product may first be released in a limited segment and tested in the real business environment (UAT- User acceptance testing).
- Then based on the feedback, the product may be released as it is or with suggested enhancements in the targeting market segment.
- After the product is released in the market, its maintenance is done for the existing customer base.

UNIT I:

- **Introduction:** Evolution, Software development projects, Exploratory style of software developments, Emergence of software engineering, Notable changes in software development practices, Computer system engineering.
- **Software Life Cycle Models:** Basic concepts, Waterfall model and its extensions, Rapid application development, Agile development model, Spiral model.

Waterfall model and its extensions

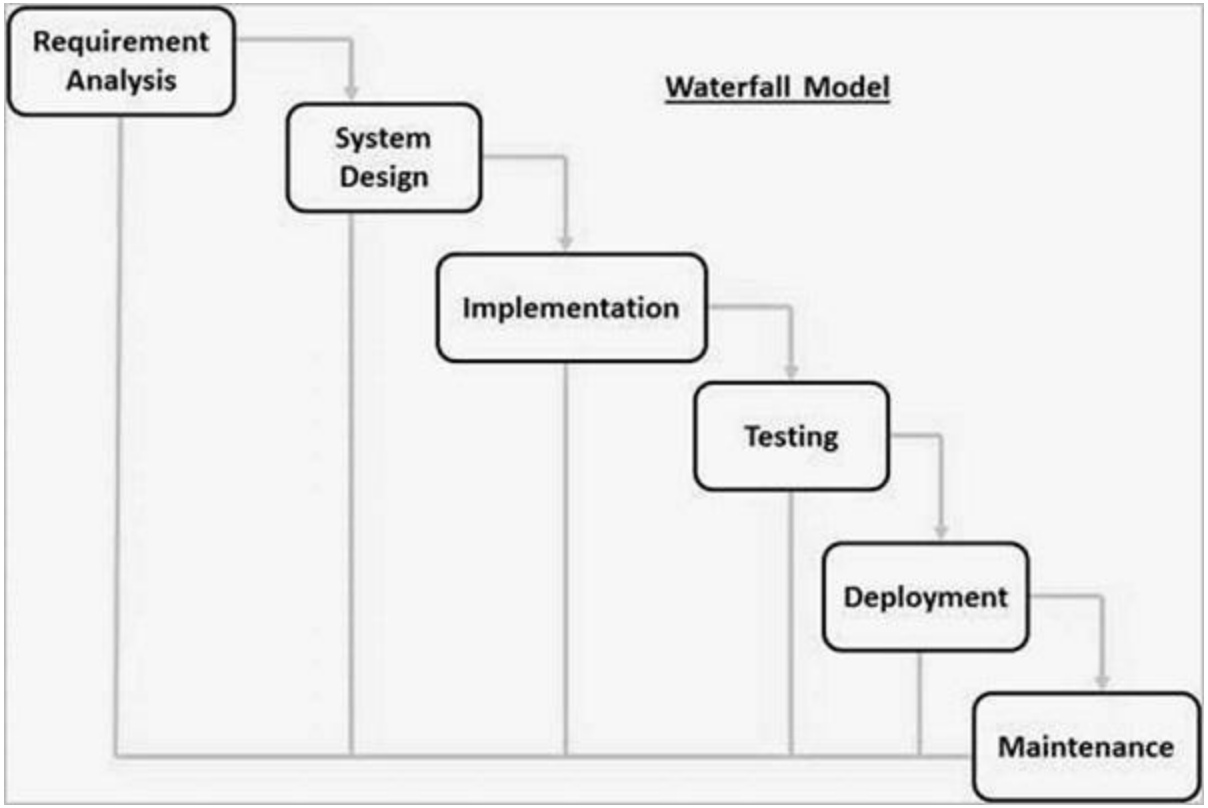
- Waterfall Model
- V Model
- Incremental Process Model
- Prototype Model

Waterfall model and its extensions

- Waterfall Model
- V Model
- Incremental Process Model
- Prototype Model

Waterfall Model

Waterfall Model



Waterfall Model

Waterfall Model model is most suited are:

- When the requirements are constant and not changed regularly.
- A project is short
- Where the tools and technology used is consistent and is not changing
- Product definition is stable.

Waterfall Model

Advantages of the Waterfall Model are as follows:

- Simple and easy to understand and use.
- Phases are processed and completed one at a time.
- Works well for smaller projects where requirements are very well understood.
- Well understood milestones.
- Easy to arrange tasks.
- Process and results are well documented.

Waterfall Model

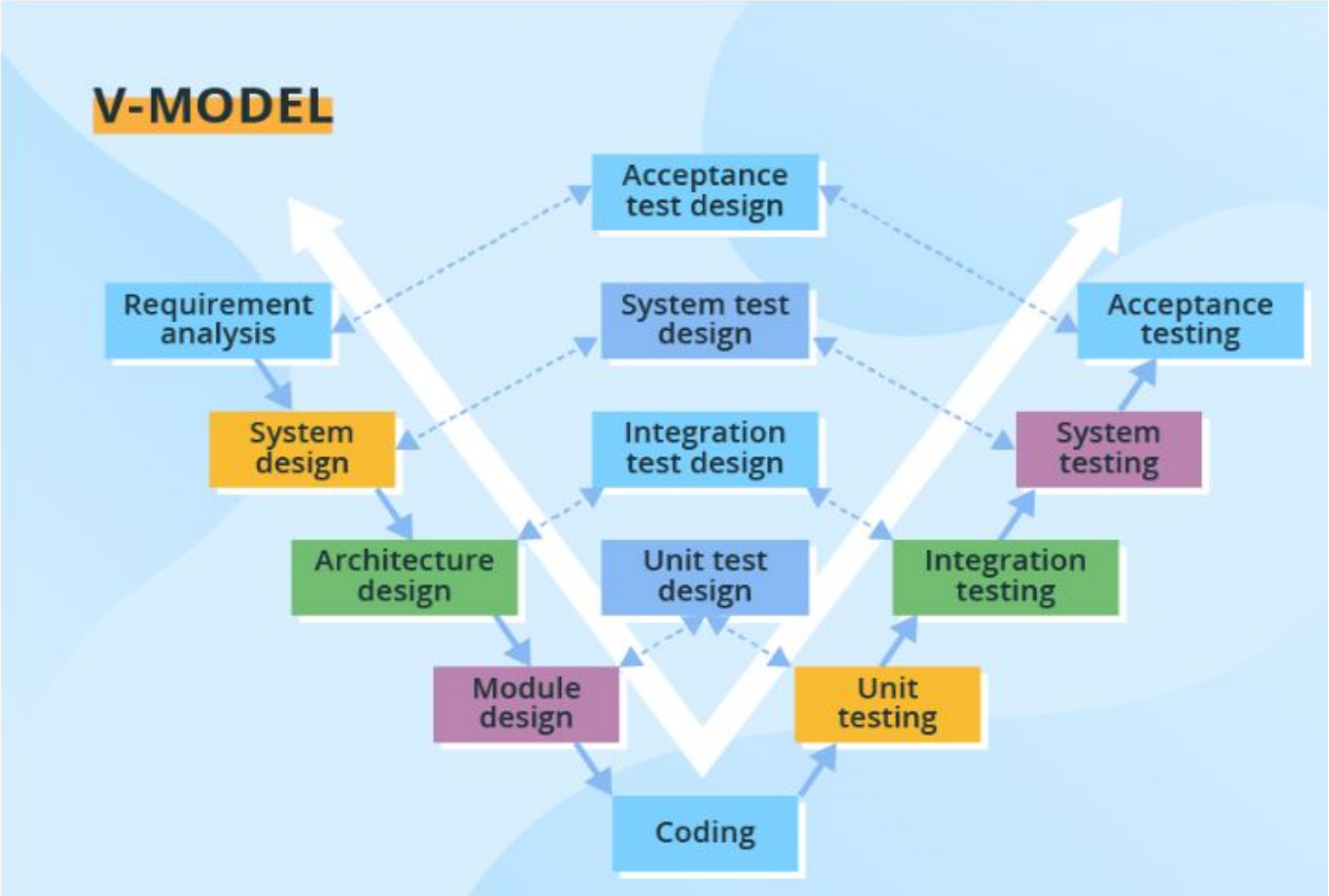
The major disadvantages of the Waterfall Model are:

- No working software is produced until late during the life cycle.
- High amount of risk and uncertainty.
- Not a good model for complex and object-oriented projects.
- Poor model for long and ongoing projects.
- Cannot accommodate changing requirements.

Waterfall model and its extensions

- Waterfall Model
- V Model
- Incremental Process Model
- Prototype Model

V Model


V-MODEL
Acceptance test design
Requirement analysis
System test design
Acceptance testing
System design
Integration test design
System testing
Architecture design
Unit test design
Integration testing
Module design
Unit testing
Coding

V Model

- **Verification:** It involves static analysis technique (review) done without executing code.
- It is the process of evaluation of the product development phase to find whether specified requirements meet.

V Model

- **Validation:** It involves dynamic analysis technique (functional, non-functional), testing done by executing code.
- Validation is the process to evaluate the software after the completion of the development phase to determine whether software meets the customer expectations and requirements.

V Model

Verification	Validation
It includes checking documents, design, codes and programs.	It includes testing and validating the actual product.
Verification is the static testing.	Validation is the dynamic testing.
It does *not* include the execution of the code.	It includes the execution of the code.
Methods used in verification are reviews, walkthroughs, inspections and desk-checking.	Methods used in validation are Black Box Testing, White Box Testing and non-functional testing.
It checks whether the software conforms to specifications or not.	It checks whether the software meets the requirements and expectations of a customer or not.
It can find the bugs in the early stage of the development.	It can only find the bugs that could not be found by the verification process.

Waterfall model and its extensions

- Waterfall Model
- V Model
- Incremental Process Model
- Prototype Model

Incremental Model

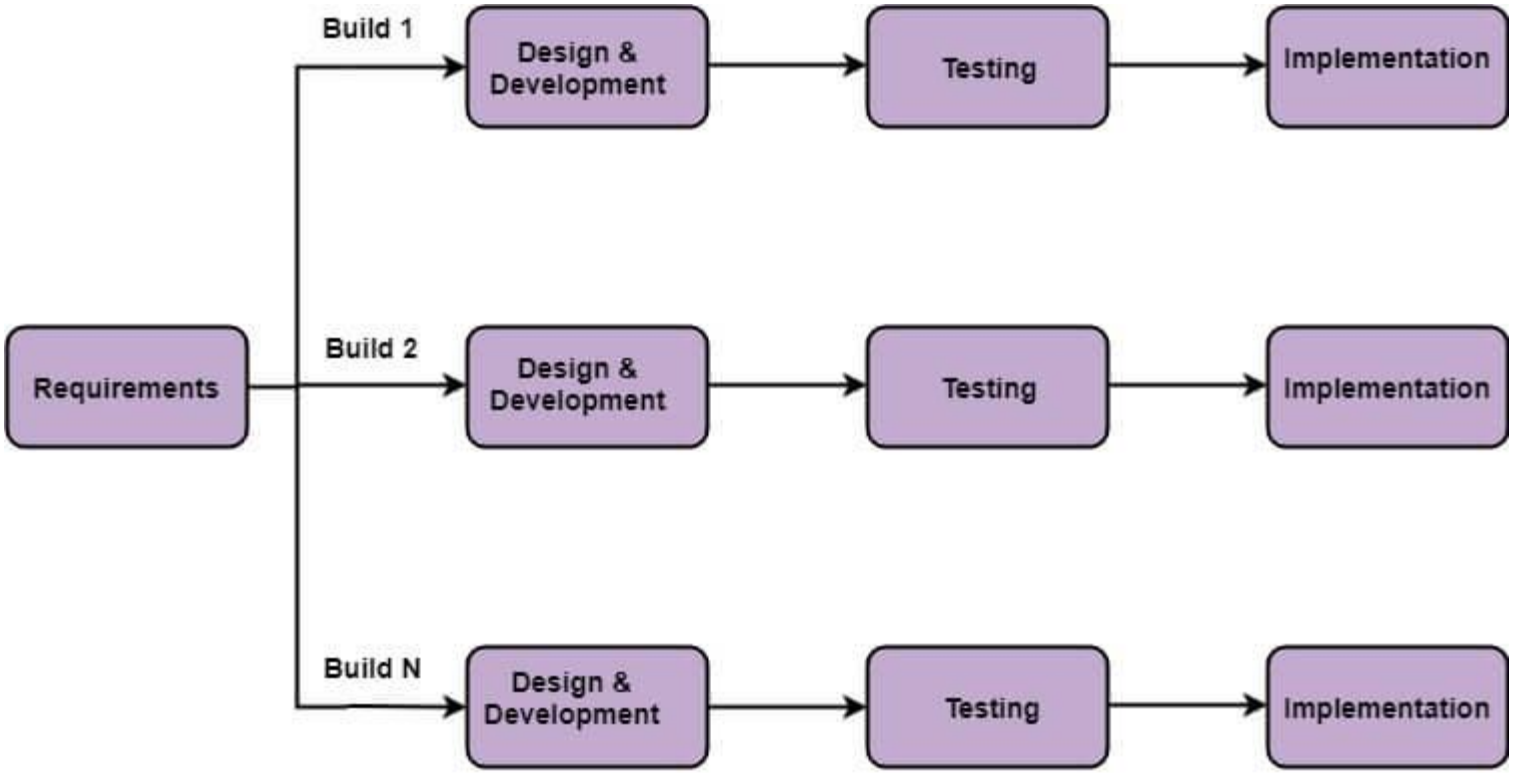


Fig: Incremental Model

Incremental Model

When to use the Incremental model:

- This model can be used when the requirements of the complete system are clearly defined and understood.
- Major requirements must be defined; however, some details can evolve with time.
- There is a need to get a product to the market early.

Incremental Model

Advantages of Incremental model:

- Generates working software quickly and early during the software life cycle.
- This model is more flexible.
- It is easier to test and debug during a smaller iteration.
- In this model customer can respond to each built.
- Easier to manage risk because risky pieces are identified and handled during it'd iteration.

Incremental Model

Disadvantages of Incremental model:

- Needs good planning and design.
- Needs a clear and complete definition of the whole system before it can be broken down and built incrementally.
- Total cost is higher than **waterfall**.

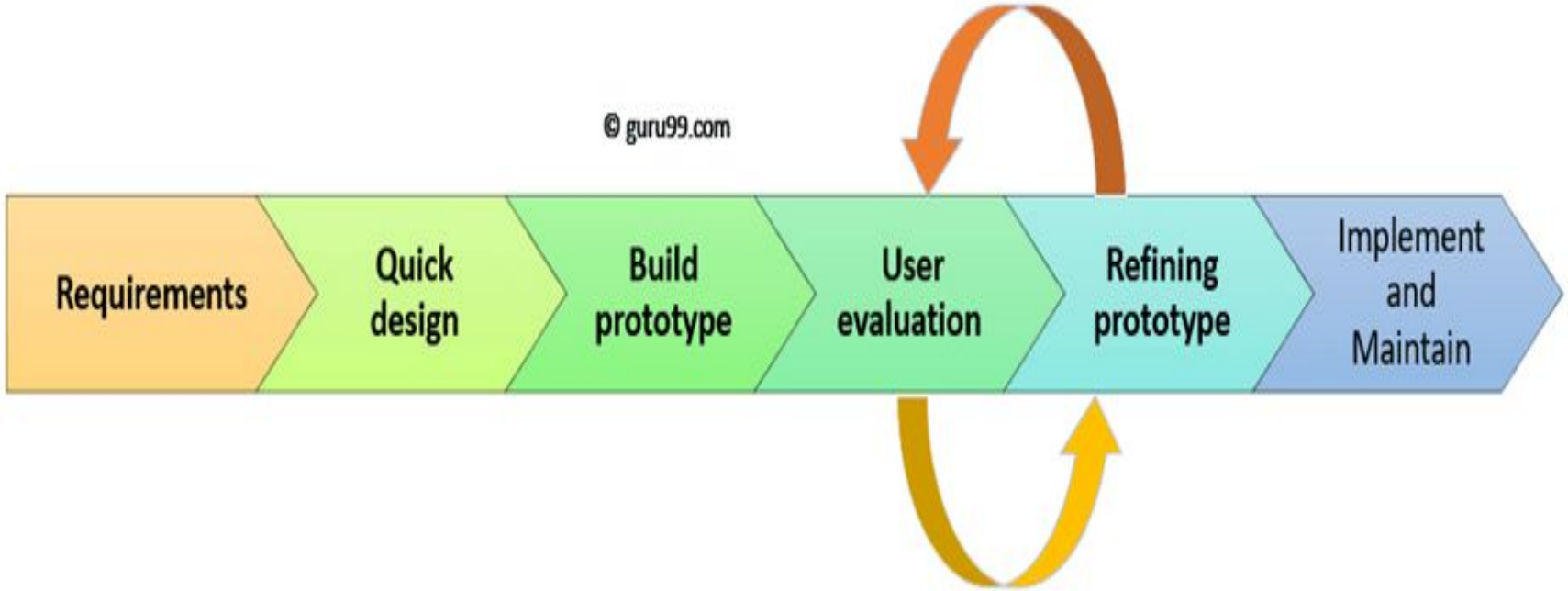
Waterfall model and its extensions

- Waterfall Model
- V Model
- Incremental Process Model
- Prototype Model

Prototype Process Model

- **Prototyping Model** is a software development model in which prototype is built, tested, and reworked until an acceptable prototype is achieved.
- It also creates base to produce the final system or software.
- It works best in scenarios where the project's requirements are not known in detail.
- It is an iterative, trial and error method which takes place between developer and client.

Prototype Process Model



Prototype Process Model

Advantage of Prototype Model

- Reduce the risk of incorrect user requirement
- Good where requirement are changing/uncommitted
- Support early product marketing
- Reduce Maintenance cost.

Prototype Process Model

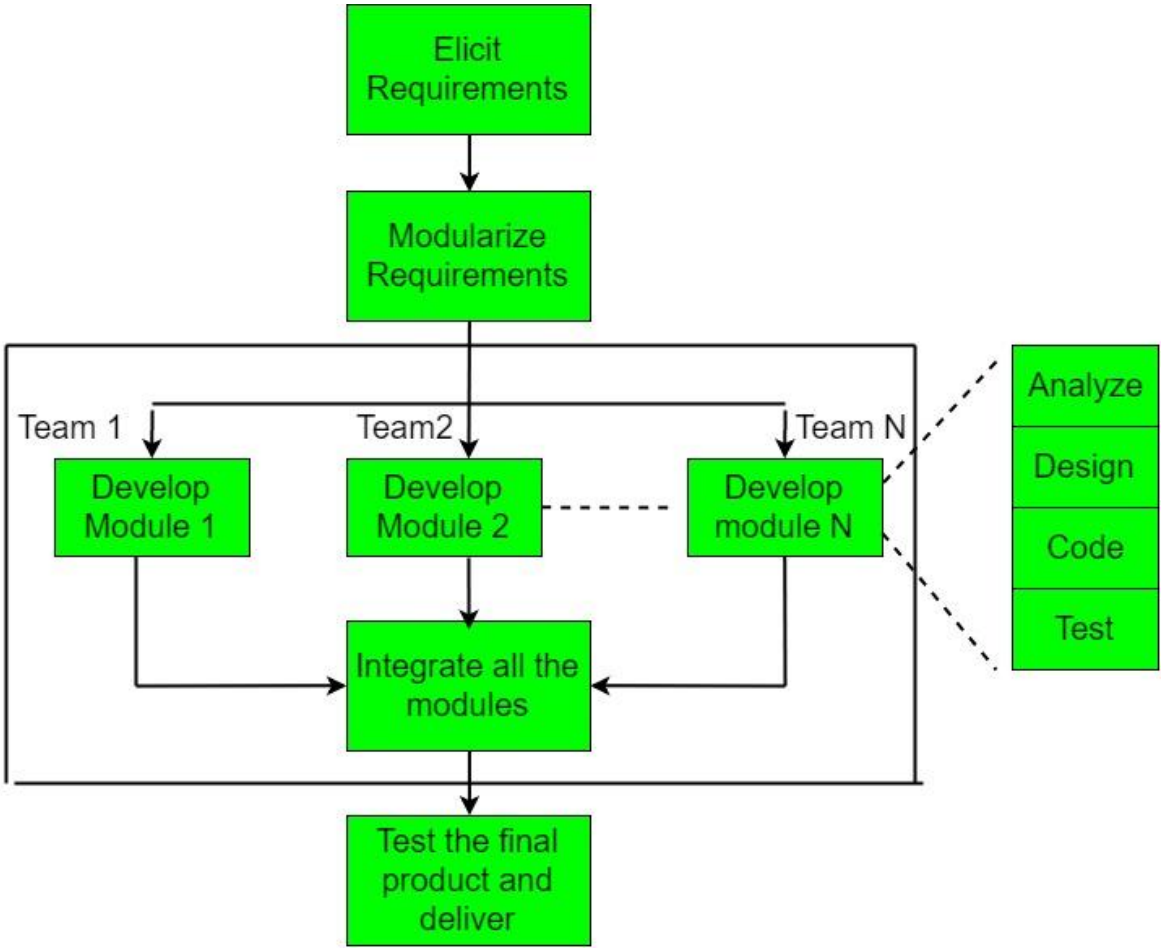
Disadvantage of Prototype Model

- Require extensive customer collaboration
- Easy to fall back into the code and fix without proper requirement analysis, design, customer evaluation, and feedback.
- Special tools & techniques are required to build a prototype.

UNIT I:

- **Introduction:** Evolution, Software development projects, Exploratory style of software developments, Emergence of software engineering, Notable changes in software development practices, Computer system engineering.
- **Software Life Cycle Models:** Basic concepts, Waterfall model and its extensions, Rapid application development, Agile development model, Spiral model.

Rapid application development



Rapid application development

- The RAD model or Rapid Application Development model emphasizes quick and iterative release cycles, primarily focusing on delivering working software in shorter timelines.
- IBM first proposed the Rapid Application Development or RAD Model in the 1980s.
- The RAD model is a type of incremental process model in which there is a concise development cycle.
- The RAD model is used when the requirements are fully understood and the component-based construction approach is adopted.

Rapid application development

- A software project can be implemented using this model if the project can be broken down into small modules wherein each module can be assigned independently to separate teams.
- These modules can finally be combined to form the final product.
- Development of each module involves the various basic steps as in the waterfall model i.e. analyzing, designing, coding, and then testing.
- Another striking feature of this model is a short period i.e. the time frame for delivery(time-box) is generally 60-90 days.

Rapid application development

- When to use the RAD Model?
- Well-understood Requirements: When project requirements are stable and transparent, RAD is appropriate.
- Time-sensitive Projects: Suitable for projects that need to be developed and delivered quickly due to tight deadlines.
- Small to Medium-Sized Projects: Better suited for smaller initiatives requiring a controllable number of team members.
- Low technological Complexity: Suitable for tasks using comparatively straightforward technological specifications.

UNIT I:

- **Introduction:** Evolution, Software development projects, Exploratory style of software developments, Emergence of software engineering, Notable changes in software development practices, Computer system engineering.
- **Software Life Cycle Models:** Basic concepts, Waterfall model and its extensions, Rapid application development, Agile development model, Spiral model.

Agile development model

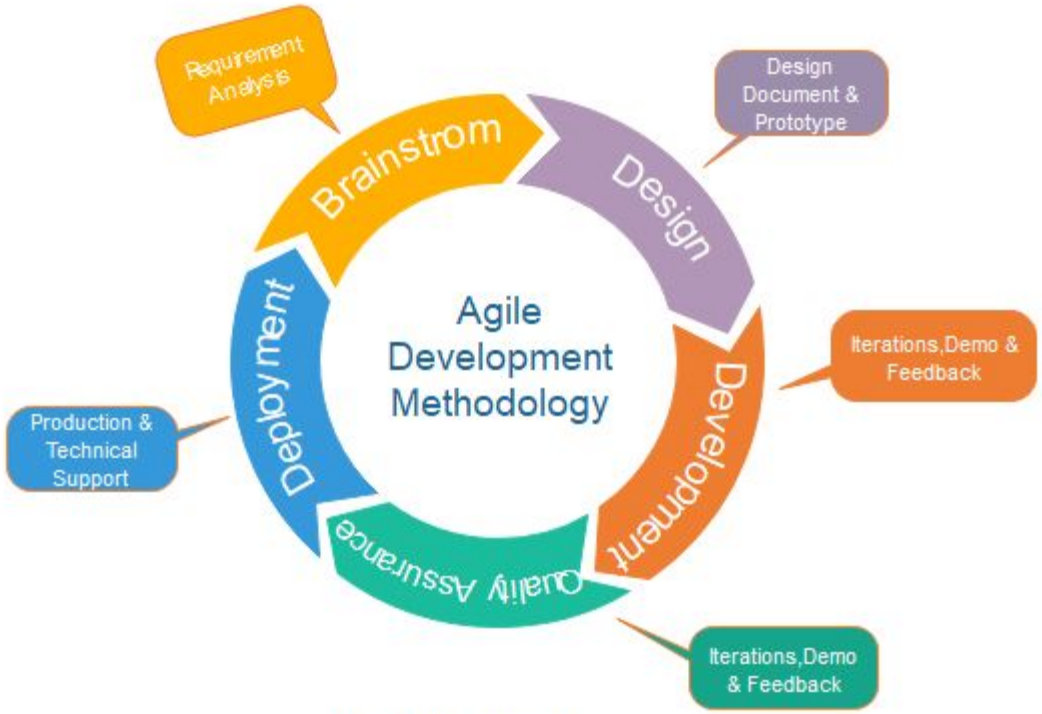


Fig. Agile Model

Agile development model

- Agile SDLC model is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product.
- Agile Methods break the product into small incremental builds. These builds are provided in iterations.
- Each iteration typically lasts from about one to three weeks.

Agile development model

Advantage(Pros) of Agile Method:

- Frequent Delivery
- Face-to-Face Communication with clients.
- Efficient design and fulfils the business requirement.
- Anytime changes are acceptable.
- It reduces total development time.

Agile development model

Disadvantages(Cons) of Agile Model:

- Due to the shortage of formal documents, it creates confusion and crucial decisions taken throughout various phases can be misinterpreted at any time by different team members.
- Due to the lack of proper documentation, once the project completes and the developers allotted to another project, maintenance of the finished project can become a difficulty.

Extreme Programming ****

- Extreme Programming (XP) is an agile software development framework that aims to produce higher quality software, and higher quality of life for the development team.
- The five values of XP are communication, simplicity, feedback, courage, and respect.

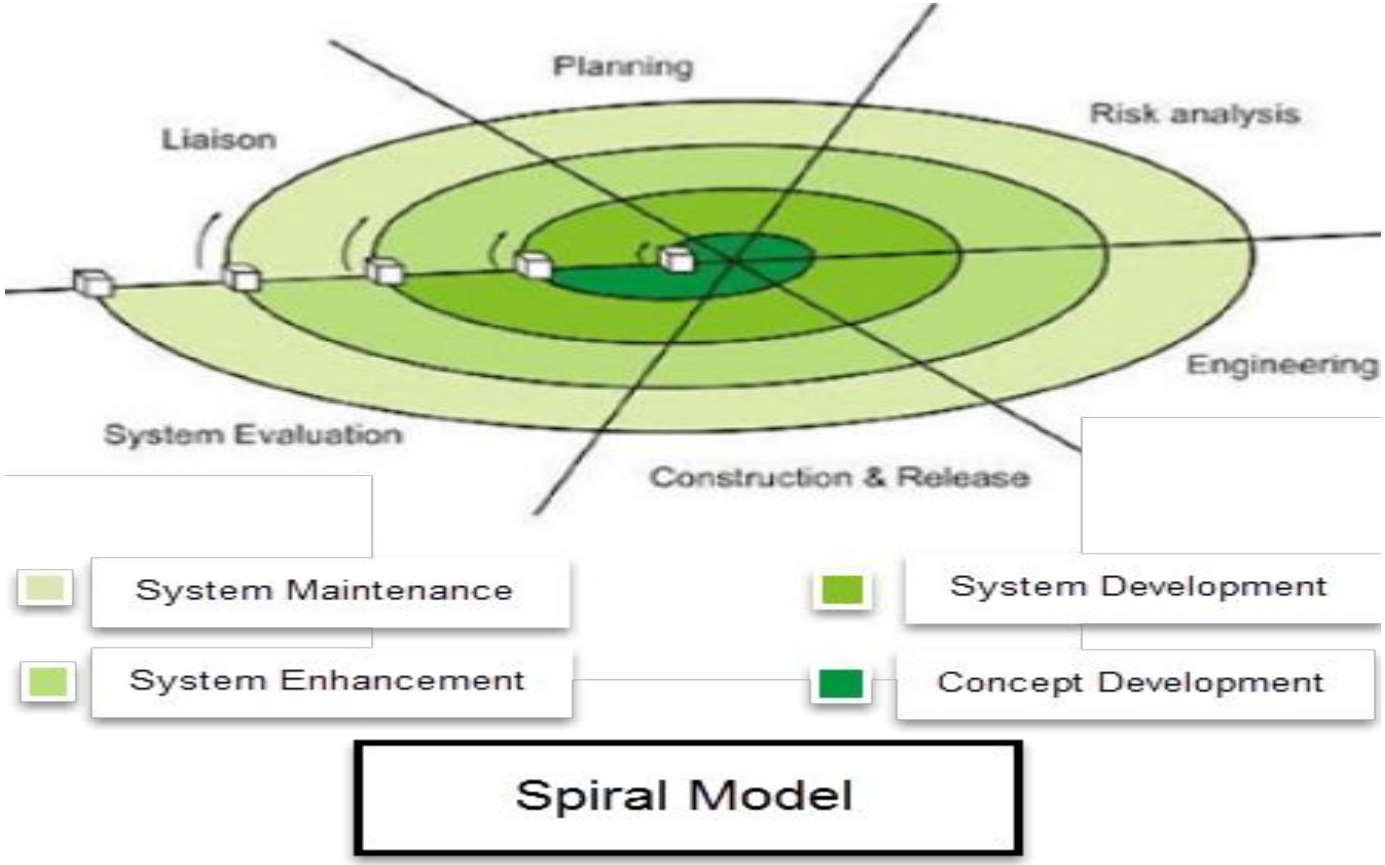
UNIT I:

- **Introduction:** Evolution, Software development projects, Exploratory style of software developments, Emergence of software engineering, Notable changes in software development practices, Computer system engineering.
- **Software Life Cycle Models:** Basic concepts, Waterfall model and its extensions, Rapid application development, Agile development model, Spiral model.

Spiral Model

- The development process in Spiral model in SDLC, starts with a small set of requirement and goes through each development phase for those set of requirements.
- The software engineering team adds functionality for the additional requirement in every-increasing spirals until the application is ready for the production phase.

Spiral Model



Spiral Model

Uses of a Spiral Model:

- When there is a budget constraint and risk evaluation is important.
- For medium to high-risk projects.
- Customer is not sure of their requirements which is usually the case.
- Requirements are complex and need evaluation to get clarity.
- Significant changes are expected in the product during the development cycle.

Spiral Model

The advantages of the Spiral SDLC Model are:

- Changing requirements can be accommodated.
- Requirements can be captured more accurately.
- Development can be divided into smaller parts.

Spiral Model

The disadvantages of the Spiral SDLC Model are:

- Not suitable for small or low risk projects and could be expensive for small projects.
- Process is complex.
- Large number of intermediate stages requires excessive documentation.